

Physical activation functions (PAFs): An approach for more efficient induction of physics into physics-informed neural networks (PINNs)

Jassem Abbasi^{*}, Pål Østebø Andersen

Department of Energy Resources, University of Stavanger, 4036, Norway

ARTICLE INFO

Keywords:

Deep Learning
Physics-informed Neural Networks
PDEs
Activation Functions

ABSTRACT

In recent years, the evolution of Physics-Informed Neural Networks (PINNs) has reduced the gap between Deep Learning (DL) based methods and analytical/numerical approaches in scientific computing. However, there are still complications in training PINNs and the optimal interleaving of physical models. In this work, we introduce the concept of Physical Activation Functions (PAFs), in which one can use generic AFs with their mathematical expression inherited from the physical description of the evaluated system, instead of solely usage of standard activation functions (AFs) such as *tanh*, and *sigmoid* for all the neurons. The expression of PAFs could be selected based on individual terms appearing in the analytical solution, the initial or boundary conditions of the PDE system, or a component in the composition-of-functions type solutions. The PAFs could be applied in NNs, either in explicit, or self-adaptive forms. In the explicit approach, the main activation function of the network is replaced by PAF in some of the neurons of the network. In the self-adaptive implementation approach, the relative impact of PAFs (compared to the base AF) for each neuron was determined automatically. We tested the performance of PAFs in both forward and inverse problems for several PDEs, such as 1D and 2D wave equations, the Advection-Convection equation, the 1D heterogeneous, and 2D diffusion equations, and the Laplace equation. The main advantage of PAFs, compared to using standard AFs, was the more efficient constraining and interleaving of PINNs with the physical phenomena and their underlying mathematical models. The added PAFs significantly improved the predictions of PINNs for the testing data that were out-of-training distribution. Furthermore, applying PAFs reduced the size of the PINNs by up to 75 % in different cases while maintaining the same accuracy. Also, the training process was improved by reducing the value of the total loss term by one to two orders of magnitude. Furthermore, it improved the precision of the calculated properties in the examined inverse problems, for both clean and noisy observational data. It can be concluded that using PAFs helps in generating PINNs with less complexity and more validity for longer ranges of prediction.

1. Introduction

The capabilities of machine learning (ML) in different fields such as computer vision, and language processing have been proven in recent years. In consequence, the concept of scientific ML (SciML) has been introduced to take advantage of ML in scientific problems [12,28]. One of the main techniques of SciML is the Physics Informed Neural Networks (PINNs) that try to fuse the governing laws of the investigated systems (that may be in the forms of Ordinary Differential Equations (ODEs) or Partial Differential Equations (PDEs)) to Neural Networks (NNs). The PINNs methodology tries to train NNs that can approximately reflect the physical behavior of the problem (usually in a spatial-temporal domain), including its initial and boundary conditions

[31]. There is hope that progress in the PINNs techniques will lead to more interpretable and reliable SciML models, especially in dealing with scientific problems where the accuracy and flexibility of the model are critical [20]. The PINNs are trained by enforcing the NNs to follow the mathematical representation of the involved physics and the provided measured/calculated datasets (in the case of availability, such as in inverse problems). In recent years, many studies have applied PINNs in technical fields such as mechanics [29], fluid dynamics [5,16], power systems [27], and geosciences [1,2,11,24].

In general, embedding physics into the NNs is performed by three pathways: observational data, physical loss functions, or mathematical induction [17]. Observational data is the traditional approach for training ML models in which the model is learned to accurately

^{*} Corresponding author.

E-mail address: jassem.abbasi@uis.no (J. Abbasi).

<https://doi.org/10.1016/j.neucom.2024.128352>

Received 6 January 2023; Received in revised form 26 March 2024; Accepted 10 August 2024

Available online 14 August 2024

0925-2312/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

interpolate between the data that ought to reflect the underlying physical principles. This is the weakest form of interleaving physics into the model. The second approach, physical loss functions, is added as a soft condition to the training algorithm to yield forecasts roughly satisfying a given set of physical constraints (for instance, their underlying differential equations). The mathematical induction fuses the physics by tailored interventions in the model architecture, commonly expressed in the form of physics-informed mathematical constraints, for instance, encoding the bounds of the system, symmetries in the dynamics, or equilibrium conditions [9]. However, this approach may apply to limited possible applications [17].

The mathematics behind PINNs originates from the Universal Approximation Theorem [13], which states that NNs can closely approximate any continuous function with only one hidden layer and a finite number of neurons. However, although the Universal Approximation Theorem guarantees the existence of a neural approximator for the given mathematical problem, it does not offer any procedure for selecting the best network architecture or training the NN [6]. Many researchers dealing with PINNs reported that minimizing the loss functions during a training operation is challenging, especially when the complexity of the corresponding differential equations increases [23,26,35]. To deal with this difficulty, numerous approaches were suggested. Bora et al. [4] and Krishnapriyan et al. [21] utilized a sequence-to-sequence approach where the data from the space-time domain is imported into the model partially during the training process. Also, they recommended the Curriculum Regularization approach where the complexity of the target loss function (e.g., the conditions that the PINN should meet) is increased during the learning.

Some of the studies on the improvement of PINNs focused on the modification of activation functions (AFs) of the neurons in the model. The choice of AFs in NNs may significantly impact training efficiency and model performance [32]. Different studies, such as Jagtap et al. [14] and Jagtap et al. [15], introduced the concept of a locally adaptive activation function. In this approach, a scalable parameter is added to each layer/neuron that avoids the gradient descent optimization approaches attracted to local minimums. Various studies also tried to introduce special network architectures to combine different mathematical equations and NNs [25,30,36]. To establish more physics constrained NNs, some studies also suggested their co-utilization with symbolic regression approaches [7,22]. However, there are still many challenges in developing authentic PINNs for scientific computations, particularly extrapolation to the out-of-training-distribution data [18].

In this work, a new concept called Physical Activation Function (PAF) will be introduced, which can be considered a mathematical induction approach to passing physics into ML models. This approach helps make the PINNs more constrained to the expected physical/mathematical behaviour. It will be shown that using this concept can substantially reduce the NNs size and improve the training process of PINNs. Also, it improves the extrapolation capabilities of PINNs in dealing with out-of-training-distribution data (i.e., predicting in domains that are out of the training range). In the following, the theoretical background of the concept will be introduced. Then the advantages of the approach will be demonstrated by various examples.

2. Mathematical definition

In scientific ML, we are interested in finding unknown variables from known variables. Here, we consider that the relationship between known vs. unknown variables is representable by differential equations (ODE or PDEs). Given y as the unknown variable of interest as a function of known variables $x_1, \dots, x_d$ and t , the mathematical equation of the physics is represented in the format of ODE or PDE, with the general form of:

$$\mathcal{F}(x, y) = 0 \quad \text{in } \Omega,$$

$$\mathcal{B}(x, y) = 0 \quad \text{on } \partial\Omega$$

$$\mathcal{I}(x, y) = 0 \quad \text{in } \Omega, \quad t = 0 \quad (1)$$

defined on the domain $\Omega \subset \mathbb{R}^d$ with the boundary domain $\partial\Omega$, and d is the spatial dimension of the system. Here, $x := [x_1, \dots, x_d; t]$ indicates the spatial-temporal coordinate vectors, and $\mathcal{F}$ is the differential operator. $\mathcal{I}$ and $\mathcal{B}$ denote the arbitrary initial and boundary (e.g., Dirichlet, Neumann, and Robin) conditions of the problem, respectively. In the following, the mathematical representation of NNs, PINNs as well as the concept of Physical Activation Function (PAF) is introduced.

2.1. Overview of neural networks

A fully connected feedforward NNs, or a Multi-layer Perceptron (MLP), is a long sequence of units (neurons) connected with simple linear operations. In a generic NN called $\mathcal{N}$, there are $\mathcal{D}$ layers ($\mathcal{D} - 2$ hidden layers). In the k_{th} layer, there are n_k neurons. Each neuron receives an input, x_{k-1} , from the previous layer that is linearly transformed [14]:

$$\ell(x_{k-1}) \triangleq w_k x_{k-1} + b_k; k = 1, 2, \dots, \mathcal{D} \quad (2)$$

The network weights ($w_k \in \mathbb{R}^{n_k \times n_{k-1}}$) and biases ($b_k \in \mathbb{R}^{n_k}$) are trainable parameters (Fig. 1). The output of each neuron is passed through an activation function, $\sigma(\bullet)$, and then sent to the next layer as:

$$\ell(x_{k-1}) = \sigma(w_k x_{k-1} + b_k); k = 1, 2, \dots, \mathcal{D} \quad (3)$$

Then the final output of the NN is mathematically calculated from the inputs as:

$$\mathcal{N}(x; \theta) = \sigma(w_1 \sigma(\dots \sigma(w_2 \sigma((w_1 x + b_1) + b_2) \dots) + b_l) \quad (4)$$

where, $\theta = \{w_k, b_k\}$ represents the set of trainable parameters in the NN. The number of parameters in set θ is N_θ . Using a different mathematical representation:

$$\mathcal{N}(x; \theta) = (\ell_D \circ \ell_{D-1} \dots \circ \ell_1)(x) \quad (5)$$

where $\circ$ denotes the function decomposition, e.g., for sample functions $f(x)$ and $g(x)$ it is written as:

$$(f \circ g)(x) = f(g(x)) \quad (6)$$

Fig. 1 shows the graphical representation of a fully connected feedforward NN. In the training of the NN with a set of $\{x, y\}$ dataset, the goal is finding a set of θ^* such that the defined loss function is minimized:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{j=1}^N \mathcal{L}(y_j, \mathcal{N}(x_j; \theta)) \quad (7)$$

where N is the number of data points, and $\mathcal{L}$ is the loss function. Here, $\mathcal{L}$ is the mean squared error (MSE) between the true and predicted values:

$$\mathcal{L}(y_i, \mathcal{N}(x_i; \theta)) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathcal{N}(x_i; \theta))^2 \quad (8)$$

The trainable parameters (θ) are optimized to minimize the loss function in an iterative way. In each iteration (epoch), the θ set is updated as follows:

$$\theta^{m+1} = \theta^m - \eta_l \nabla_{\theta} \mathcal{L}(\theta) \quad (9)$$

where m and η_l denote the epoch number and the optimizer's learning rate, respectively. The training is carried out on training data, and to avoid underfitting or overfitting, the model is checked on separate validation data during the training process. The training process is stopped when the error of validation data starts increasing.

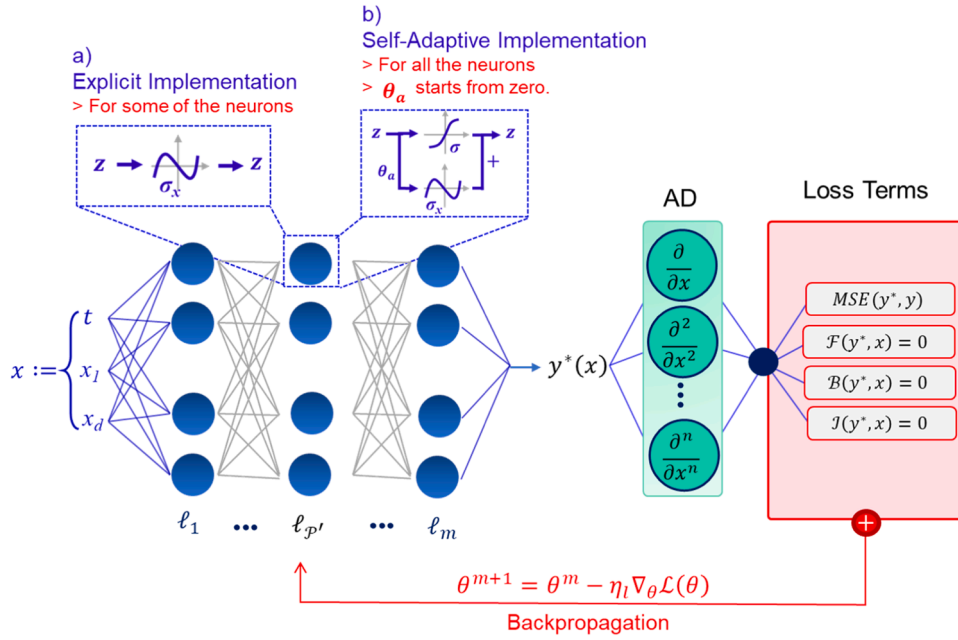


Fig. 1. Simplified architecture of a PINN with inputs $x := [x_1, \dots, x_d, t]$ and output $y^*(x)$, as well as two different suggested implementation strategies: a) Explicit Implementation, b) Self-adaptive implementation (SA-PAF). Here, the PINN is an MLP with ℓ_m layers. In the explicit implementation strategy, all the neurons in a layer may not have the same AFs. The differential of y^* versus input variables are calculated using Automatic Differentiation (AD). The loss terms of PDE ($\mathcal{F}$) and initial ($\mathcal{I}$) and boundary ($\mathcal{B}$) conditions are calculated at each epoch. Here, $\sigma_x(z)$ is an arbitrary PAF defined in the layer ℓ_p . Also, z is the input to the neuron from the previous layer.

2.2. Overview of physics informed neural networks

A typical PINN tries to take benefits from observational data and physical loss functions. According to the Universal Approximation Theorem of NNs, for any sufficiently regular unknown variable y , there is a large enough NN, $\mathcal{N}$, with trainable parameters θ , such that:

$$y(x) \approx y^*(x) = \mathcal{N}(x; \theta^*), \quad x \in \Omega \quad (10)$$

where $y^*(x)$ is the output of the NN. Given the known governing PDE in Eq. (1):

$$\mathcal{F}(y^*; v) \approx 0 \quad \text{in } \Omega,$$

$$\mathcal{B}(y^*; v) \approx 0 \quad \text{on } \partial\Omega$$

$$\mathcal{I}(y^*; v) \approx 0 \quad \text{in } \Omega, \quad t = 0 \quad (11)$$

Eqs. (10) and (11) suggest that an arbitrary PDE can be solved by replacing the unknown $y(x)$ with a neural network $y^*(x) = \mathcal{N}(x; \theta)$ into the constituent PDE and finding the optimum trainable parameters θ^* such that $\mathcal{F}(\mathcal{N}; v) \approx 0$, $\mathcal{B}(\mathcal{N}; v) \approx 0$ and $\mathcal{I}(\mathcal{N}; v) \approx 0$ (see Fig. 1). The derivatives of $\mathcal{N}$ versus the input parameters are calculated using Automatic Differentiation (AD) tools that are available in all modern DL libraries such as TensorFlow and PyTorch. Formally, we need to assess the validity of $\mathcal{N}$ in meeting the conditions in Eq. (11). So, a loss function called physical loss function ($\mathcal{L}_p$) is defined in the domain Ω :

$$\mathcal{L}_p(\theta) = \lambda_1 \int_{\Omega} |\mathcal{F}(\mathcal{N}; v)|^2 dx + \lambda_2 \int_{\partial\Omega} |\mathcal{B}(\mathcal{N}; v)|^2 dx + \lambda_3 \int_{\Omega} |\mathcal{I}(\mathcal{N}; v)|^2 dx \quad (12)$$

where λ_1 , λ_2 , and λ_3 denote the weight factors of each loss term. The mentioned loss weights should be determined by trial and error, or by utilizing self-adaptive approaches [38]. The distribution of datasets and fidelity of the model have determining roles in the selection of the optimum λ_i values [8]. We wish to find the θ^* which minimizes the $\mathcal{L}_p(\theta)$. This loss term enforces the NN into learning the physics of the problem. If $\mathcal{L}_p(\theta^*) \approx 0$, by definition, the NN is a solution to the differential equation (i.e., $y(x) \approx \mathcal{N}(x; \theta^*)$), or a neural surrogate to the analytical model [41]. Consequently, this NN is called Physics Informed

Neural Networks (PINNs). Fig. 1 shows the way that the loss terms are used to train a symbolic PINNs model by the backpropagation algorithm (Eq. (9)).

2.3. Physical activation functions

Suppose for the Eq. (1), $\mathcal{F}(x, y) = 0$, there is an operator $y = \mathcal{P}(x)$ in the investigating domain Ω that approximates the solution of the problem. We assume the operator $\mathcal{P}(x)$ can be defined as multivariate composition-of-functions type function [3], such that:

$$y(x) = \mathcal{P}(x) = \mathcal{P}_{1,1}(\mathcal{P}_{2,1}(\mathcal{P}_{i,2}, \mathcal{P}_{i,1}, \dots, \mathcal{P}_{i,j}), \mathcal{P}_{2,2}, \dots, \mathcal{P}_{2,j})(x) \quad (13)$$

where $\mathcal{P}_{ij}(x)$ is a subfunction or argument for the general function $\mathcal{P}(x)$, and takes $\mathcal{P}_{i-1,1,j}$ as inputs. It assists in decomposing the main function into simpler and smaller components. Suppose the function $p_x = \mathcal{P}_x(x)$ is a subfunction in above equation and induces some parts of the behaviour of the physical phenomena under study. It may represent system in the whole or limited range of the whole domain, and also may demonstrate the exact or approximate behaviour of the investigating system. Where x is the input variable(s), including both spatial and temporal variables, and p_x is the output of the function.

Remembering Eq. (10), for collocation points $\{x\}$ in domain Ω , say that there is a NN or PINNs, $\mathcal{N}$, where $y^*(x) = \mathcal{N}(x; \theta^*)$, such that:

$$y(x) \approx y^*(x) \text{ for } x \in \Omega \quad (14)$$

Here, $\mathcal{N}(x; \theta)$ would be the neural surrogate of $\mathcal{P}(y, x)$ in domain Ω . According to Eq. (5) and similar to Eq. (13), $y^*(x) = \mathcal{N}(x; \theta^*)$ can be written as the below sequence of operations:

$$y^*(x) = \ell_{1,1}(\ell_{2,1}(\ell_{i,2}, \ell_{i,1}, \dots, \ell_{i,j}), \ell_{2,2}, \dots, \ell_{2,j})(x) \quad (15)$$

where ℓ_{ij} is also a subfunction, or sub-network in the main NNs $\mathcal{N}$. In this work, we suggest replacing the non-trained subfunction ℓ_x with the already known subfunction $\mathcal{P}_x$:

$$z_k^* = \mathcal{P}_x(z_k) \quad (16)$$

Here, z_k is defined as the input values to the arbitrary hidden layer k from the hidden layer $k-1$:

$$z_k = w^k /^{k-1} + b^k \quad (17)$$

We believe the suggested replacement helps in reducing the complexity of the network, and also constraints the network during the optimization. So, we can rewrite:

$$y^*(x) = \ell_{1,1}(\ell_{2,1}(\mathcal{P}_x, \ell_{i,1}, \dots, \ell_{i,j}), \ell_{2,2}, \dots, \ell_{2,j})(x) \quad (18)$$

To make the suggested replacement compatible with the current approaches in NNs, we implement the function $\mathcal{P}_x$ into the network as an activation function, i.e., σ_x . We call this function a Physical Activation Function (PAF). In this work, we suggest substituting the operations carried out by the routine activation functions (e.g., *tanh*, and *sigmoid*) with the physical activation function σ_x . Fig. 1 illustrates how the σ_x may be placed in a (PI)NN. The assumptions, possible applications, and recommended approaches for choosing a suitable PAF are summarized in Table 1. The criteria for selecting suitable PAFs for various problems are further elaborated and discussed in the following sections, accompanied by different examples.

2.4. Implementation

In this work, we suggest two different implementation strategies for the PAF.

2.4.1. Hard Implementation

Considering the computations in layer k of the network can be considered as the summation of the outputs of the computations in all the neurons, we can write:

$$z_k = \sum_{i=1}^n \sigma(z_{k-1}) \quad (19)$$

As a hard implementation approach, we can replace the PAF with one or more than one of the neurons in layer k . So, we can rewrite:

$$z_k = \sum_{i=1}^{n-n_x} \sigma(z_{k-1}) + \sum_{j=1}^{n_x} \sigma_x(z_{k-1}) \quad (20)$$

Where n is the total number of neurons in layer k , and n_x is the number of neurons with PAF.

Table 1

A summary of the assumptions, and other details in choosing the right PAF, as well as implementing them in the PINNs.

Main Requirements
The PAF function should be:
- Continuous - Differentiable - Non-constant
Preferably:
- PAFs can be allocated to a single neuron, a group of neurons, or a hidden layer(s). - The defined PAFs should preferably require low computational costs. - Although we used fully connected feedforward NNs here, PAFs can be implemented in other network architectures, such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs).
Implementation
- Hard implementation - Self-adaptive implementation (SA-PAF)
How to choose the PAFs expression:
- It may be defined based on the equations representing the initial or boundary conditions (e.g., Dirichlet, Neumann, and Robin) of the investigated system. - Considering the decomposed form of the investigated system, it may be a component of the partial differential equation being solved. - It can be a part of the analytical solution (or any other type of solutions) of the system in the lower dimensions, or lower complexity forms; the defined PAF function should partially regenerate the trends in the physical behavior of the problem.

2.4.2. Self-Adaptive Implementation

As the second approach, we suggest utilizing a self-adaptive (SA-PAF) approach to choose between the best combination of the activation functions [37], instead of manually applying them in the network. We want to give each neuron the ability to pick the best combination between the base AF (σ), e.g., *tanh*, and the specific PAFs. This way, we hope to make the PINNs optimization process more reliable and smoother. Assuming $\mathcal{P}_{x,i}(z)$ as the candidate physical activation functions, the best AF for each neuron may be calculated by:

$$\mathcal{P}(z) = \sigma(z) + \sum_{i=1}^N \theta_{a,i} \mathcal{P}_{x,i}(z) \quad (21)$$

where θ_a is the trainable parameter which determines the relative weight of the PAF in each neuron, and N is the total number of the candidate PAFs. The θ_a is set to a starting value of zero and are allowed to change to in any directions during the training. The implementation of θ_a may be neuron-wise, or layer-wise. In this work, we implemented the neuron-wise approach.

2.5. Training of PAF-PINNs

Considering that the backpropagation algorithm is the most determining tool in training the PINN, the impact of PAFs on the automatic differentiation process is also essential. The partial derivative of $y^*(x)$ versus x in Eq. (18) gives:

$$\left. \frac{\partial y^*(x)}{\partial x} \right|_x = \left. \frac{\partial y^*}{\partial \ell_1} \right|_{\ell_1} \cdot \left. \frac{\partial \ell_1}{\partial \ell_2} \right|_{\ell_2} \cdot \dots \cdot \left. \frac{\partial \mathcal{P}_x}{\partial z} \right|_z \cdot \dots \cdot \left. \frac{\partial \ell_m}{\partial x} \right|_x \quad (22)$$

In this NN, the PAF $\mathcal{P}$ is a part of the layer $\ell_{\mathcal{P}}$. Since $\mathcal{P}$ is inspired by the physical behavior of the investigating phenomenon, its derivative also can be considered physical. This is an advantage when we are dealing with automatic differentiation algorithms. Since $\partial \mathcal{P}_x / \partial z$ is analytically representing the behavior of the system, less attempt is required for training based on the differentials of the PINN model versus input variables. It needs to be considered that the PAF needs to be continuous and differentiable in the investigating domain.

2.6. Computations

All the mathematical models were implemented in Python 3.9.7 environment. The models were developed on top of the PyTorch 1.10 library as well as Pandas, Matplotlib, and NumPy libraries. Training of NNs was performed using Adam [19] or LBFGS (Low-memory Broyden–Fletcher–Goldfarb–Shanno algorithm) optimizers [34] also to compare the performance of different optimizing algorithms.

3. Results

In the following, the concept of Physical Activation Function is examined in different case studies. In these cases, generally, two loss terms were used. The basic loss term, named ‘data-based loss’ is the MSE of the NN (or PINN) predictions vs. the observed true values, i.e., Eq. (8). The second set of loss terms is called the ‘physical loss’, where the discrepancy between the governing equation (plus any acting constraints, e.g., initial conditions or boundary conditions), and the PINNs approximation is evaluated and minimized during the training. The mathematical expression of this loss term is shown in Eq. (12). So, for PINN $\mathcal{N}$ and the collocation points $\{x, y\}$, with the mathematical expression $\mathcal{F}(x, y)$, boundary condition $\mathcal{B}(x, y)$, and initial condition $\mathcal{I}(x, y)$, the ‘total loss’ term is expressed as:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (\mathcal{N} - y)^2 + \frac{1}{N} \sum_{i=1}^N |\mathcal{F}(x, \mathcal{N})|^2 + \frac{1}{N} \sum_{i=1}^N |\mathcal{B}(x, \mathcal{N})|^2 + \frac{1}{N} \sum_{i=1}^N |\mathcal{I}(x, \mathcal{N})|^2 \quad (23)$$

where in the right-hand side of this equation, the first term is the data-based loss ($\mathcal{L}_d$), and the summation of the following terms is the physical loss ($\mathcal{L}_p$) that are expressions for the squared errors of the PDE, boundary conditions, and initial conditions, respectively. In the following section, the impact of PAFs on forward problems is studied.

3.1. Forward problems

3.1.1. A component of analytical solution functions as PAF (1D Harmonic Oscillator)

Here we show the potential of using PAFs in a simple case with an analytical solution, i.e., a mechanical harmonic oscillator. Also, an analysis of the implementation of PAFs was carried out. The dynamics of movements in mechanical oscillators such as springs or pendulums is a classic problem of interest in mechanics. The position (χ) of the oscillating object with respect to time (t) is written as:

$$\mathcal{F}(\chi, t) = m \frac{\partial^2 \chi}{\partial t^2} + \mu \frac{\partial \chi}{\partial t} + k\chi = 0 \quad (24)$$

where m is the mass, k the spring force constant, and μ a constant that parameterizes the strength of the damping. This PDE has an exact analytical solution for a damped oscillation ($\delta < \omega_0$):

$$\chi(t) = e^{-\delta t} (2A \cos(\phi + \omega t)); \text{ with } \omega = \sqrt{\omega_0^2 - \delta^2} \quad (25)$$

when $\chi(t=0) = 1$. Here, $\phi = \tan^{-1}(-\delta/\omega)$, $\delta = \mu/2m$, $\omega_0 = \sqrt{k/m}$, and $A = 1/2\cos(\phi)$.

We train an NN that can regenerate the solution of the above PDE (its exact analytical form). To do so, we prepared a set of collocation points $\{t, \chi\}$, and a physical loss function related to the PDE. Two loss functions of $\mathcal{L}_d$ and $\mathcal{L}_p$ were combined linearly, and the total loss function was imported to an Adam optimizer. In this example, the exact analytical solution of the PDE is available. As Eq. (25) shows, the mathematical behavior of a generic harmonic oscillator is analogous to the multiplication of $\cos x$ and e^x functions. Based on the analytical solution, two PAFs were defined.

$$\sigma_{\cos}(z) = \cos(z) \quad (26)$$

$$\sigma_{\exp}(z) = \exp(z) \quad (27)$$

In this context, z is the neuron’s input from the posterior layer. It should be mentioned that the applied PAFs did not include the meta-parameters in the analytical solution.

Before investigating the impact of PAFs, we tested the ability of PINNs in the accurate prediction of harmonic oscillation versus time, especially for the out-of-training datasets. Fig. 2 shows the results. The dashed lines are from the analytical model. The purple points are training data used for the minimization of $\mathcal{L}_d$ (data-based loss). The region where each loss term was active is shown in each plot in Fig. 2. The output of the NN for the training range is shown in blue, while the testing data (out of range of the training data) is shown in red. The structure of the NNs and AFs used for their neurons are shown in each figure. In all the NNs, different types of AFs (i.e., *linear* and *tanh* AFs) were used.

Fig. 2a and b show the NN output for two different training ranges; neither case used the physical loss function during training. In both cases, the NN could follow the correct trend only in the training range. In Fig. 2c, a PINN model is utilized. Data were available in the range of 0–0.75 s, and $\mathcal{L}_p$ was added in the range 0–1.5 s. Adding $\mathcal{L}_p$ could improve the predictions in the range of 0.75–1.50 s, where the data was not available (in comparison to Fig. 2a). This indicates that PINN can compensate for the lack of data in the training range. However, the main question is whether the trained PINN can also work well for the out-of-training-distribution ranges of data. For all the cases in Fig. 2, the models failed even in predicting the trend of the testing data, demonstrating that adopting PINNs in traditional PDE solvers still requires more attention. In Fig. 2d, utilizing the periodic AF (sinusoidal) suggested by Ziyin et al. [40], i.e., $\sigma(z) = z + \sin^2(z)$, could not significantly improve the results.

Then, we tried to assess the performance of the PAFs defined in Eqs. (31) and (32)). The data ranges are the same in the following cases, summarized in Table 2 and Fig. 3. As shown in Table 2, using PINNs instead of traditional NNs could reduce the total loss ($\mathcal{L}_d + \mathcal{L}_p$) from 6.9e-4–4.8e-4 (2.3e-4 for adaptive *tanh*). However, for the testing data (out-of-training-distribution prediction), R^2 was negative for both cases (far from the expected values). For PINNs with PAFs, the loss was reduced by two orders of magnitude, and the R^2 of testing data (extrapolation range) was improved significantly towards 0.97. The comparison of Fig. 2 and Fig. 3 shows these differences clearly. We can see that the extrapolation of the PAF-PINNs is almost accurate up to $t=2.5$ s, though the error in the calculations gradually increases over time (after $t>3.0$ s, the predictions will be unreliable).

Furthermore, by a trial-and-error approach, we searched for the smallest possible size of NNs for each case. Our observations showed that when a PAF was added to the network, the model could be trained with fewer parameters (in this example, θ reduced from 246 to 67). These results indicate that smaller NNs with fewer trainable parameters are possible to be used when a PAF is included in the NN.

3.1.2. A component of analytical solutions as a PAF (1D Advection-Dispersion Equation)

In this case, we show the feasibility of defining PAFs based on a component of the analytical solution of the problem. Hydrodynamic dispersion in a semi-infinite porous media is a common phenomenon in many subsurface processes, such as solute transport in aquifers and chemical flooding in oil recovery. The below PDE is yielded by combining advection and dispersion transport processes (one-dimensional and homogeneous system).

$$\frac{\partial C}{\partial t} = D \frac{\partial^2 C}{\partial x^2} - v \frac{\partial C}{\partial x} \quad (28)$$

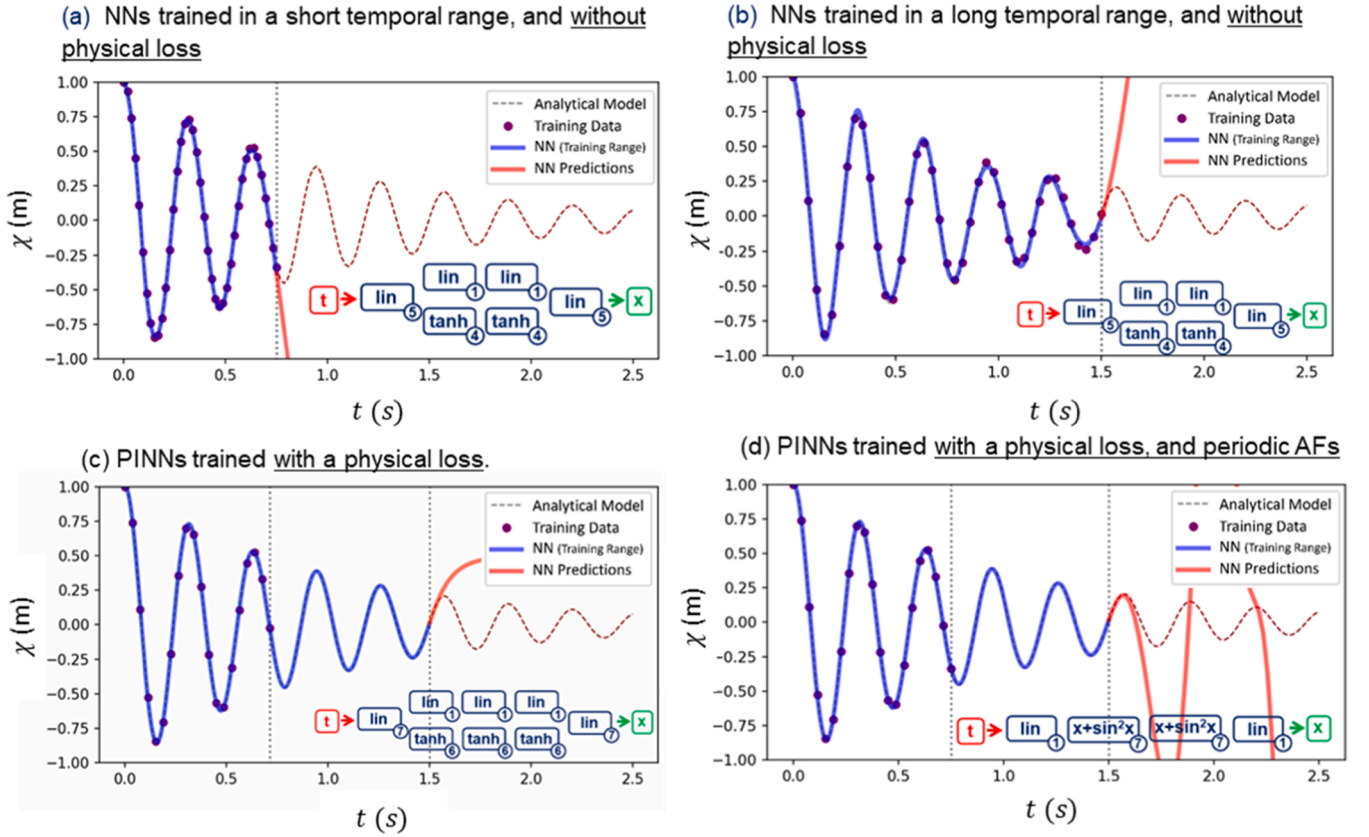


Fig. 2. The capabilities of NNs and PINNs in the prediction of the harmonic oscillation out of the training range. The architecture of the NNs is shown in each case (for simplicity, the connections between neurons are hidden). Purple points are the training data (labeled as ‘Data’), and the dash-line is the analytical solution of the PDE. The range that the physical loss is used is labeled as ‘Physics’. The output of NN in the training range is shown in blue, and its predictions for the testing data are shown in red.

Table 2

A summary of the investigated scenarios with different loss-function and activation functions for the mechanical harmonic oscillator PDE.

PINNs	PAF	Activation Functions	Training range (s)		Total Loss (m ²)	Trainable Parameters (θ)	R ²	
			Data-based	Physics-based			Train	Test
No	No	Linear, tanh	0.0–1.5	-	6.9e-4	106	0.99	-63.74
No	No	Linear, tanh	0.0–0.75	-	2.5e-4	106	0.99	-1230.2
Yes	No	tanh	0.0–0.75	0.0–1.5	4.8e-4	246	1.00	-28.12
Yes	No	Adaptive tanh	0.0–0.75	0.0–1.5	2.3e-04	246	1.00	-2052
Yes	No	$z + \sin^2 z$	0.0–0.75	0.0–1.5	1.1e-4	67	1.00	-131
Yes	Yes	$\tanh, \sigma_{\cos}, \sigma_{\exp}$	0.0–0.75	0.0–1.5	5.2e-6	67	1.00	0.98
Yes	Yes	$\tanh, \sigma_{\cos}$	0.0–0.75	0.0–1.5	6.9e-6	67	1.00	0.97

where C is the concentration of particles (mol/kg), v is fluid flow velocity (m/s), and D is the hydrodynamic dispersion coefficient (m²/s). The analytical solution for the above PDE is [39]:

$$C(x, t) = \frac{C_0}{2} \left[\operatorname{erfc} \left(\frac{x - vt}{2\sqrt{Dt}} \right) + \exp \left(\frac{vx}{D} \right) \operatorname{erfc} \left(\frac{x + vt}{2\sqrt{Dt}} \right) \right] \quad (29)$$

by the condition that:

$$C(x, 0) = 0 \quad x \geq 0$$

$$C(0, t) = C_0 \quad t \geq 0$$

$$C(\infty, t) = 0 \quad t \geq 0 \quad (30)$$

In this case, we intend to train a PINN that can predict the advection-dispersion phenomena as a surrogate model for Eq. (28). Based on the analytical solution, we defined two different physical activation

functions (PAFs) as below:

$$\sigma_{\operatorname{erfc}}(x, z) = \operatorname{erfc} \left(\frac{x - z}{2\sqrt{z}} \right) \quad (31)$$

$$\sigma_{\exp \operatorname{erfc}}(x, z) = \exp(x) \operatorname{erfc} \left(\frac{x + z}{2\sqrt{z}} \right) \quad (32)$$

where z is the input to the neuron, and x is the position. The data required for training the PINN is gathered for $C_0=1$, $v=0.003$, and $D=0.003$. The data was collected in the t range of 0–300 s, and the x range of 0–2 mm. The complete details about the ranges of data are provided in Table 3. In this example, as in previous examples, two data-based and physical losses were defined.

As a base case (with $\tanh$ AF), a PINN with a network size of $\dot{\theta} = 781$ was defined. As Fig. 4a depicts, the PINN model could perform acceptably in calculating the concentration in the training range but failed in the testing range (black lines show the PINNs results, while the colored

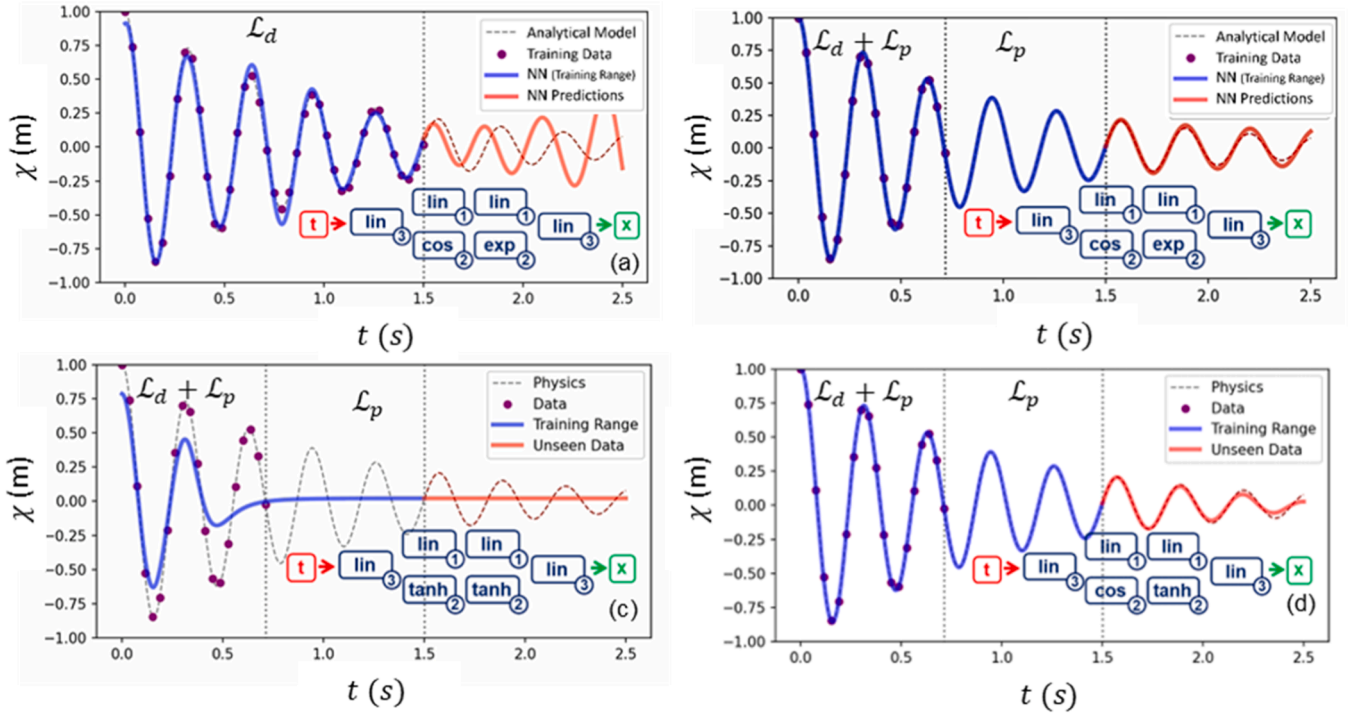


Fig. 3. The impact of PAFs on the predictions of NNs in the different scenarios. The used NNs are shown in each case (for simplicity, the connections between neurons are hidden). Purple points are the training data (labeled as ‘Data’), and the dash-line is the analytical solution of the PDE. The range that the physical loss was used is labeled as ‘Physics’. The blue line shows the training range, and the red line shows the NN predictions on the test data. a) NNs with $\sigma_{\cos}$, and $\sigma_{\exp}$ PAFs, and without physical loss; b) PINN with $\sigma_{\cos}$, and $\sigma_{\exp}$ PAFs, and with physical loss. c) PINN without PAFs and shallow hidden layers; d) PINN with $\sigma_{\cos}$ PAF, and with physical loss.

Table 3

A summary of the ranges of the dataset used for training and testing the PINN model of the advection-convection phenomenon.

Data	x-range	t-range
Data-based loss	0.0 – 0.5	0.0 – 50
Physics-based loss	0.0 – 1.0	0.0 – 100
Testing data	0.0 – 2.0	0.0 – 300

lines show the true solutions).

To test the impact of using analytical solutions as a PAF, we generated a PINNs model with a single hidden layer and only two neurons in that layer. We added the defined PAFs (Eqs. (31) and (32)) to each of the neurons. The results after training the model are shown in Fig. 4b. With only two neurons, the model could follow the real data exactly, and the optimizer could find optimum weights. The obtained results were expected because we passed the true solution of the PDE to the PINN and induced it to follow the mathematical behavior of the phenomenon. This case is very idealistic; however, this result is an indication of the theoretical advantage of using PAFs.

To extend the results to more practical cases, we tested using only one of Eqs. (31) and (32) in the PINNs (only a part of the behavior of the problem is known). As Fig. 4c and d show, these models could perform better in comparison to the base case in Fig. 4a, while N_b reduced from 781 to 49.

However, since we used a mixed combination of AFs in the layers of NNs, one may be concerned about how this mixed usage of AFs may affect the PINNs performance. In other words, it is necessary to distinguish the impacts of using a mixed combination of AFs and PAFs. To do so, we analyzed the different combinations of AFs in a PINN model with four layers, each with ten neurons. The 2nd layer of the PINN is altered in various scenarios as: 1) all the neurons used *tanh* AF, 2) used a combination of general AFs such as *tanh*, *linear*, *ReLU*, *softmax*, and

sigmoid AFs, 3) a combination of general AFs and $\sigma_{\exp, \text{erfc}}$, and 4) only using *tanh* and $\sigma_{\exp, \text{erfc}}$. The results of the different scenarios are shown in Table 4. It is shown that using a mixed combination of general AFs (that are not specific to the physics of the problem) may reduce the efficiency of the PINN model. However, as PAFs were added to the investigating layer, the total loss terms were reduced, and R^2 values for the test data were improved.

3.1.3. Initial condition function as PAF (1D Diffusion Equation)

In many practical cases, an analytical solution may not be available. In this case, we see if it is possible to use any other known functionalities between the variables as a PAF. To do so, we considered a 1D transient diffusion phenomenon with PDE:

$$\frac{\partial C}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial C}{\partial x} \right) \quad (33)$$

where C is the concentration of particles (mol/kg), and D is the diffusion coefficient (m^2/s). We numerically solved this diffusion problem using the *Py-PDE* library [42]. Using this module, we were able to choose arbitrary functions for both the initial distribution of concentration and diffusion coefficient. So, we assumed that the diffusion coefficient changes versus spatial coordinate as:

$$D(x) = 0.02(0.3x + \log(x^2 + 1.5)) \quad (34)$$

with initial condition:

$$C(x, t = 0) = 0.2 \left(\frac{1}{x^2 + 0.2} \right)^2 \quad (35)$$

and boundary conditions:

$$C(x = -\infty, t) = 0 \quad t \geq 0$$

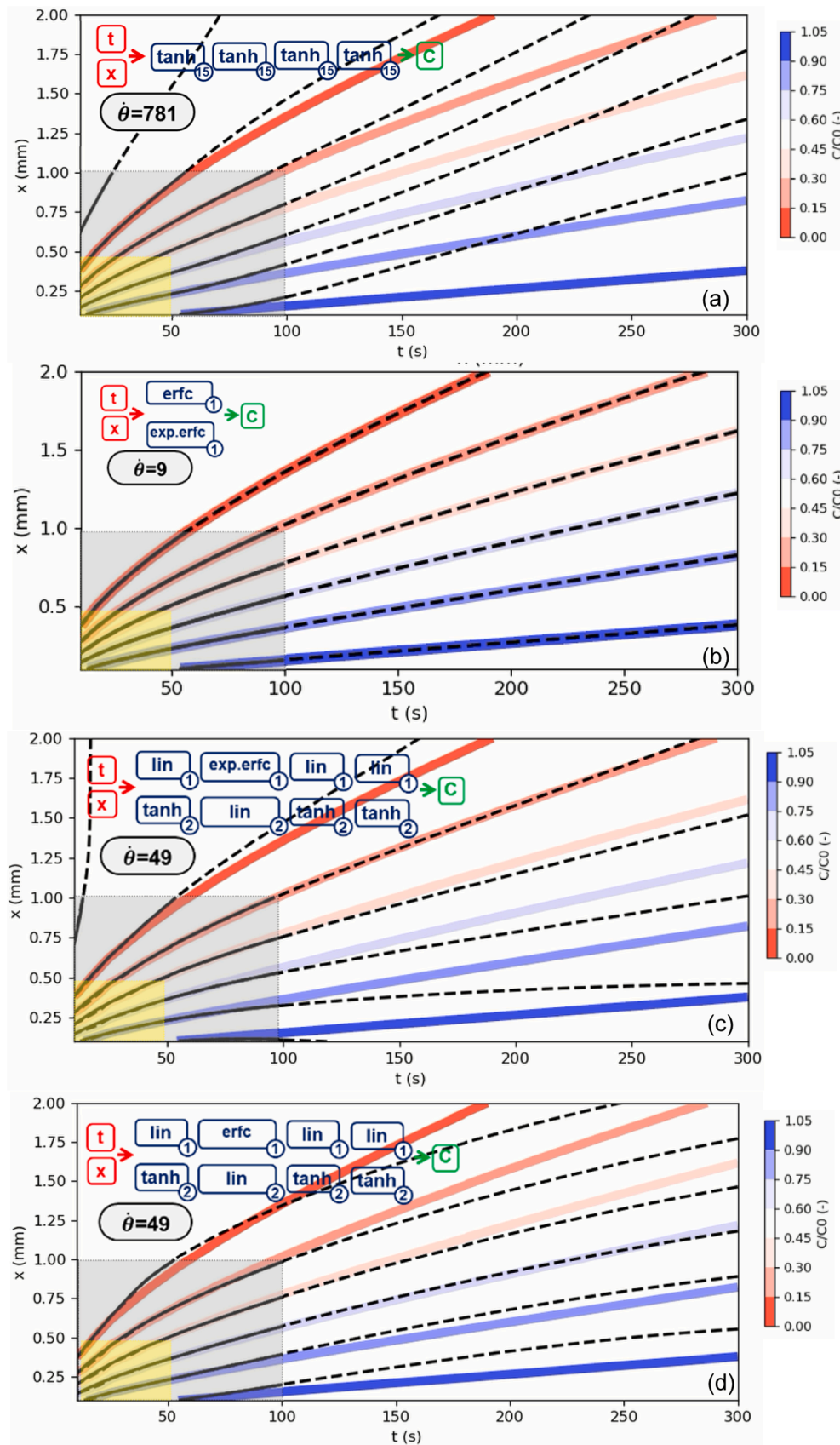


Fig. 4. Comparison of implementing different AFs in the PINN for prediction of concentration (C/C_0) in the advection-dispersion equation. In all the figures, the yellow/grey area shows the data/physics-based training range, respectively, while the remaining area is related to the testing data. The NN structure and the number of trainable parameters (θ) are shown in each figure. Black solid/dashed lines represent isolines for the NN output in training/testing regions, respectively, while the colored contour map shows the isolines of the true values of the dimensionless concentration.

Table 4

Comparing the usage of different AFs in PINNs. In all cases, four hidden layers with ten neurons were used (all with tanh AFs). In different cases, the AFs of the 2nd layer were changed in different combinations. All scenarios include 481 trainable parameters and an LBFGS optimizer was used for the training. The number in brackets shows the number of neurons for the AF.

Activation Functions [number of neurons]	Total Loss	R ²	
		Train	Test
tanh [10]	6.7e−5	0.995	0.672
tanh [2], linear[2], sigmoid[2], softmax[2], ReLU [2]	9.7e−5	0.996	0.413
tanh [5], $\sigma_{\exp, \text{erfc}}$ [2], sigmoid[1], softmax[1], ReLU [1]	5.3e−5	0.997	0.871
tanh [6], $\sigma_{\exp, \text{erfc}}$ [4],	5.2e−5	0.995	0.942

$$C(x = +\infty, t) = 0 \quad t \geq 0 \quad (36)$$

These conditions create a 1D diffusion from the point $x = 0$ to the surrounding area. The diffusion coefficient also changes versus x . The system above may be written as a composition-of-function type equation, i.e., $C(x, t) = f(x, t, D(x))$.

To train a PINN for this problem, same as in the previous cases, two different losses were defined (data-based loss and physics-based loss). Data from the numerical simulation was exported in x range of $[-1, 1]$ mm and t range $= [1, 30]$ s, with a spatial resolution of 0.1 mm and temporal resolution of 1 s. The PINN model was trained based on the data in the time range of $[0, 5]$ s and based on physics in the time range of 0–10 s. In both loss terms, the x range was -1 –1 mm. To restrict the PINN training for the times beyond the training range, the function of D versus x and the initial condition of the above problem was converted to PAFs as below:

$$\sigma_{\log x 2}(z) = (0.3z + \log(z^2 + 1.5)) \quad (37)$$

$$\sigma_{\text{inv} x 2}(z) = 0.2 \left(\frac{1}{z^2 + 0.2} \right)^2 \quad (38)$$

Then, we investigated the impact of using the PAFs in PINNs. The difference between the output of PINN with and w/o PAFs is shown in Table 5 and Fig. 5. The architecture of the PINN is shown in Fig. 5. As the table depicts, using PAFs could increase the R² of the test data for two different network widths (5 and 10). This is while for the widest network, the loss function decreased. This example reveals the potential of adding initial or boundary conditions in an explicit form to the NNs model using the concept of PAFs. This helps enforce the PINNs to follow the expected physical behavior explicitly, even in the untrained regions.

3.1.4. Composition-of-functions as a PAF (1D Heterogeneous Diffusion Equation)

In the following example, to extend the complexity of the previous diffusion problem, we held the initial concentration as Eq. (35) but altered the diffusion coefficient function from Eq. (34) to:

$$D(x) = 0.02 * \exp(- (x + \sin(0.5t))^2) \quad (39)$$

This equation makes the diffusion coefficient periodically dependent on time. The above equation lets us define the below PAFs:

$$\sigma_{\exp x}(x, z) = 0.02 * \exp(- (x + \sin(z))^2) \quad (40)$$

$$\sigma_{\exp t}(t, z) = 0.02 * \exp(- (z + \sin(t))^2) \quad (41)$$

In these equations, we derived two different PAFs from Eq. (39), where in each of them, t and x were replaced by z . We then imported these PAFs into a PINN structure. The PINN was trained in the time range of 0–15 s and tested in the time range of 15–30 s. The model was trained with the LBFGS optimizer. As depicted in Fig. 6, the PINN with PAFs could approximately follow the trend of diffusion versus time, while for the PINN case without PAFs, the predictions significantly deviated from the ground truth data, except in the training range.

In Table 6, we tried to compare the impact of using PAFs to the adaptive activation function approach [14], in terms of accuracy. It is evident that applying the PAF outperformed the adaptive activation function. This superiority was observed in terms of predictive accuracy for both the training and testing data.

3.1.5. Lower dimension solutions as a PAF (2D Wave Propagation)

Next, we test the impact of PAFs for a 2D wave propagation problem. The aim is to demonstrate whether PAFs enhance PINNs predictions in 2D problems, leveraging the mathematical insights derived from the 1D version of the problem. We can write the 2D wave propagation as:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (42)$$

when initial and boundary conditions are defined as:

$$u(t = 0, x, y) = 200$$

$$u(t, x = 0, y) = u(t, x = 1, y) = 200$$

$$u(t, x, y = 0) = u(t, x, y = 1) = 200 \quad (43)$$

For the wave propagation problem, we defined two PAFs in Sections 3–1–1 (1D harmonic oscillator), i.e., $\sigma_{\cos}$ and $\sigma_{\exp}$, where we previously showed it improved the PINNs performance for the 1D problem. In this session, we will utilize these PAFs to improve the performance of PINNs. We tried to implement different combinations of AFs using the self-adaptive approach: a) $\tanh + \tanh$, b) $\sigma_{\cos} + \sigma_{\cos}$, and c) $\sigma_{\cos} + \sigma_{\exp}$ (the first function is the base AF, the adaptive AF). The self-adaptive AF approach was applied in all the cases to make sure the total number of trainable parameters was exactly similar. With the definition of a fully connected neural network with 7 layers, each with 40 neurons, we solved the problem using a sequence of Adam and LBFGS optimizers. The training results of each case are shown in Fig. 7a. We can see the combined implementation of $\sigma_{\cos}$ (as the base AF) and $\sigma_{\exp}$ could reduce the loss term considerably compared to the usage of $\tanh$ or $\sigma_{\cos}$ AFs. b shows the mean value of θ_a term during the optimization for the third case. With proceeding in the training process, the contribution of the PAF function in the network increased. The findings in this section show how using PAFs linked to the 1D analytical solution can enhance the accuracy of PINNs when applied to solve the 2D version of the wave propagation problem.

Table 5

A summary of the investigated scenarios with different AFs for the dynamic diffusion with arbitrary initial conditions and diffusion coefficient functions.

Activation Functions	PAF	Trainable Parameters	Loss Total	R ²		
				$\mathcal{L}_d$	$\mathcal{L}_p$	Test
$\tanh$	No	111 (width: 5)	6.9e−4	6.5e−6	1.9e−4	0.972
$\tanh + \sigma_{\log x 2} + \sigma_{\text{inv} x 2}$	Yes	111 (width: 5)	7.7e−4	7.3e−6	2.3e−4	0.995
$\tanh$	No	371 (width: 10)	1.3e−4	6.3e−7	8.5e−5	0.927
$\tanh + \sigma_{\log x 2} + \sigma_{\text{inv} x 2}$	Yes	371 (width: 10)	8.7e−5	6.5e−7	3.9e−5	0.994

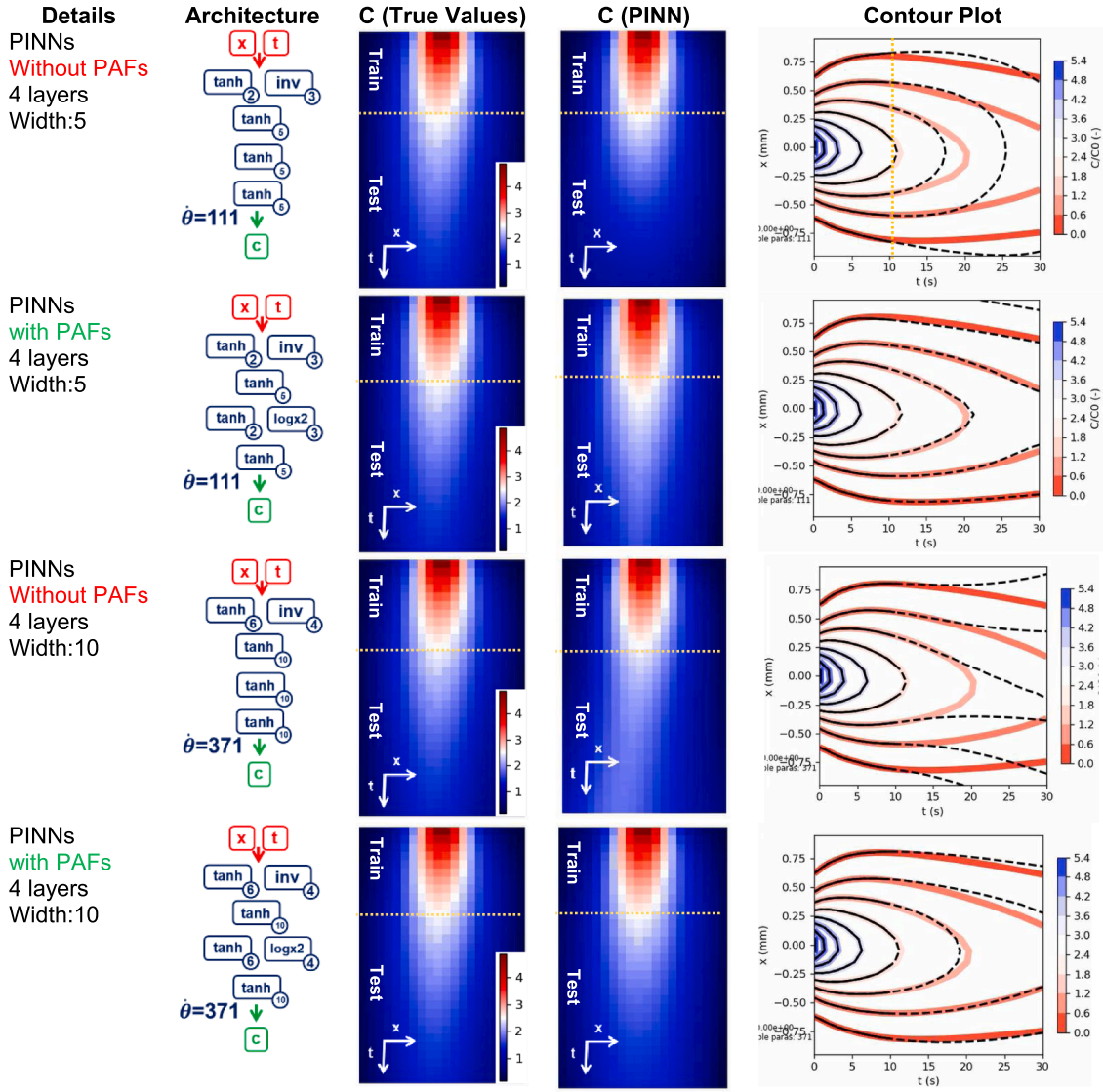


Fig. 5. 1-dimensional diffusion when D is a function of position; Comparison of the performance of PINNs before and after implementing PAFs. The first column in each row shows the details of each case. In the contour maps, the colored contour lines show the true values, the black lines show the PINNs outputs, the solid line is used on the training data range, and the dashed line is for the test data range. The LBFGS optimizer is used in this case.

3.1.6. Lower dimension solutions as a PAF (2D Heat Diffusion Equation)

Here, we utilize PAFs for improved PINNs evaluation of the 2D heat diffusion equation:

$$\frac{\partial T}{\partial t} = \kappa \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (44)$$

in which, κ is defined as the thermal diffusivity (m^2/s), and T is the local temperature. Assuming the problem to be steady state ($\frac{\partial T}{\partial t} = 0$), the boundary conditions were defined as:

$$T(x=0, y) = -\sin(y) \quad (45)$$

$$T(x=1, y) = \sin(y) \quad (46)$$

$$T(x, y=0) = 0 \quad (47)$$

$$T(x, y=1) = 0 \quad (48)$$

The 1D version of the above problem with a point-source boundary condition (Dirac-delta function) is [10]:

$$u(x, t) = \frac{1}{\sqrt{4\pi\kappa t}} \exp\left(-\frac{x^2}{4\kappa t}\right) \quad (49)$$

Considering the similarity between the 1D and 2D heat transfer problems, we may define σ_{exp} as a suitable PAF.

To compare to the solution of PINNs, the reference solution of the system was generated utilizing FEniCS [33]. Concerning the architecture of Physics-Informed Neural Networks (PINNs), a fully connected neural network comprising 7 hidden layers, each with 20 neurons, was established. To investigate the impact of PAFs, three distinct scenarios were evaluated utilizing the self-adaptive implementation approach:

1. **Base Case - tanh AF:** Employing the tanh AF as the base approach.
2. **Periodic (σ_{cos}) AF:** Applying σ_{cos} as both base AF and PAF. The usage of σ_{cos} is motivated from the applied boundary conditions of the problem.
3. **Combination of σ_{cos} and σ_{exp} AFs:** Utilization of a combination of σ_{cos} and σ_{exp} AFs. This strategy serves the purpose of satisfying both the boundary conditions and the analytical solution of the 1D version of the heat diffusion problem.

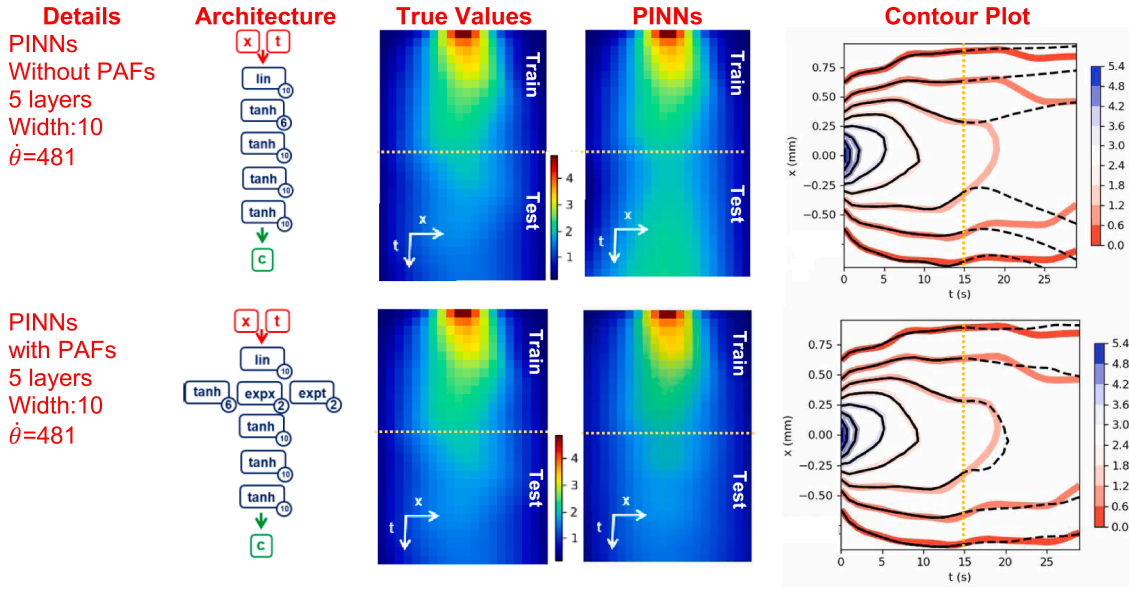


Fig. 6. 1D diffusion when the diffusion coefficient (D) is a function of both time and position; Comparison of the performance of PINN before and after implementing PAFs. The first column in each row shows the details of each case. In the contour maps (last column), the colored contours show the true values, the black lines show the PINN outlets, the solid line is for the training data, and the dashed line is for the test data.

Table 6

A summary of the investigated scenarios with different loss-function and activation functions for Diffusion equation with the periodic initial condition.

Activation Functions	Loss	Trainable Parameters	Total Loss	R^2 train	R^2 total
$\tanh$	PINN	481	$8.3e-5$	0.911	0.850
$\text{Adaptive } \tanh$	PINN	$481+6^*$	$8.1e-5$	0.915	0.881
$\tanh + \sigma_{\text{exp}} + \sigma_{\text{expt}}$	PINN	481	$8.1e-5$	0.932	0.928

*Corresponding to the trainable parameters in the adaptive activation function.

The outcomes of the PINNs solutions are depicted in Fig. 8. The inclusion of PAFs in scenarios two and three notably improved the accuracy of solutions for the heat diffusion problem. Specifically, the total loss values after training were $9.5e-5$ for the base case, demonstrating a reduction to $2.4e-5$ and $1.3e-5$ for the second and third cases, respectively. Furthermore, the MAE of the solution decreased from 0.179 in the base case to 0.122 and 0.121 for the second and third cases, respectively. This problem demonstrates the feasibility of utilizing PAFs to improve the performance of PINNs in more complex cases.

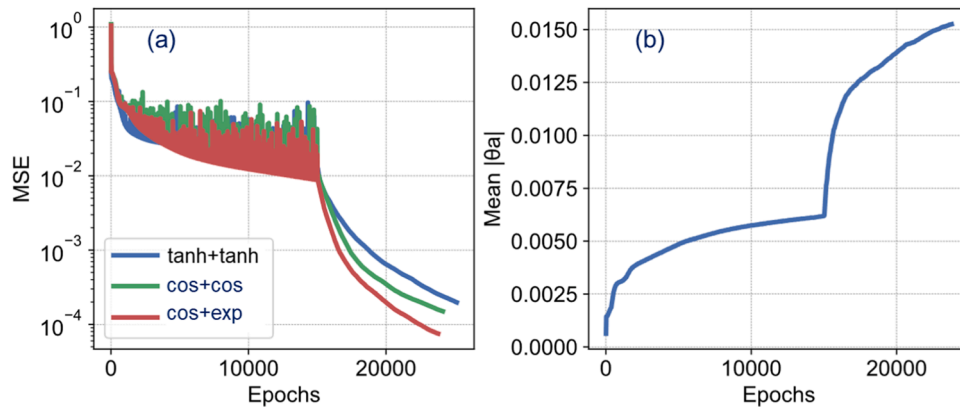


Fig. 7. The impact of adding PAFs into the network, in reducing the training loss term. Here we utilized the adaptive implementation approach. The total number of trainable parameters in all the cases was same. a) the total loss term value for networks with different AF implementations, b) the changes in the mean value of θ_a during the training.

3.2. Inverse calculations

This section examines the application of PAFs in improving two inverse PINN problems, including the diffusion equation and the 2D Laplace equation. We used the LBFGS optimization algorithm in both cases. Like in the previous examples, the network was an MLP; and to make the instances comparable (avoid initialization biases), we initialized the trainable parameters once and applied them to the different scenarios.

3.2.1. Composition-of-functions as a PAF (Diffusion Equation)

In this case, the diffusion equation is studied with the initial conditions same as Eq. (35) and the domain diffusion coefficient as a function of x :

$$D(x) = 0.01(0.30x^2 + 0.30) \quad (50)$$

As an inverse problem to estimate this function $D(x)$, we assumed that the coefficient in this equation is unknown as:

$$D(x) = 0.01(\lambda_1 x^2 + \lambda_2) \quad (51)$$

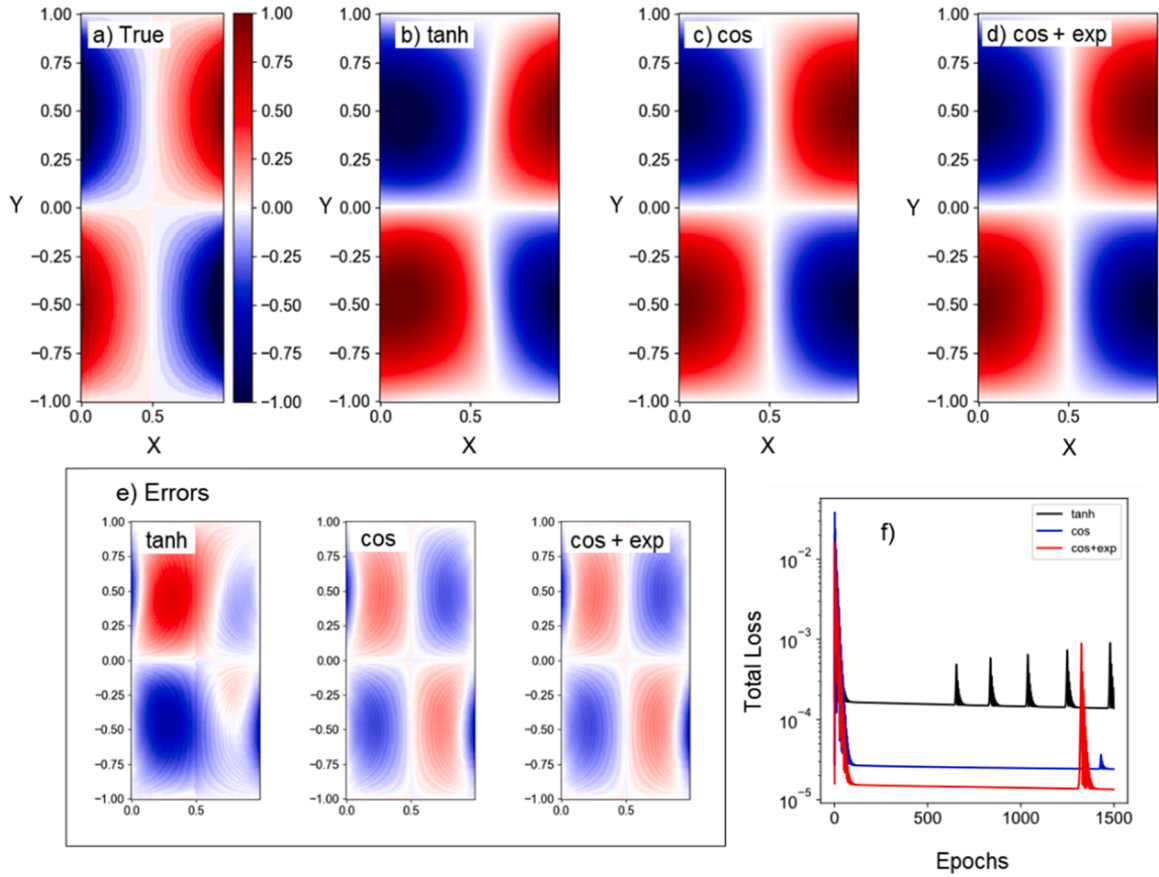


Fig. 8. The results of utilizing the different activation functions in the case of 2D heat diffusion with periodic boundary conditions. a) the true distribution of temperature, b) tanh AF, c) cos AF, d) cos and exp AFs, e) the error values for different cases, f) the total loss term corresponding different scenarios.

where true λ_1 and λ_2 are both equal to 0.30. So, in this case, we defined a PAF as:

$$\sigma_D(z, \omega_1, \omega_2) = 0.01(\omega_1 z^2 + \omega_2) \quad (52)$$

where ω_1 and ω_2 are also trainable parameters. In this experiment, we defined an MLP network with trainable parameters $\theta = \{w, b, \lambda, \omega\}$. We extracted a set of data from numerical simulations in the range of $x = (-1, +1)$ m and $t = [0, 40]$ s. In total, 82 data points were randomly chosen from the numerical simulation data to be used in the inverse calculations (see Fig. 9a). The used MLP was a 4-layer network with a depth of 9 neurons. Three different networks are investigated below:

1. **PAF_0:** An MLP with *tanh* activation function in all neurons, without PAF
2. **PAF_2:** An MLP with *tanh* activation function, except two neurons with PAF (Eq. (52)) in the second layer
3. **PAF_4:** An MLP with *tanh* activation function, except four neurons with PAF (Eq. (52)) in the second layer

Each network had 305 trainable parameters. The calculations were performed with the LR of 0.003 and 0.001, each in 8000 epochs. As it is shown in Table 7 and Fig. 9, the models with PAF had lower loss values. Also, the absolute relative errors (ARE) in the prediction of λ_1 and λ_2 reduced significantly by including PAF in the MLP model.

To investigate the effects of measurement noises on the effectiveness of PAFs, we performed inverse calculations with Gaussian noises added to the observed data (from 2.5 % to 7.5 %). We used self-adaptive PAFs ($\tanh + \sigma_D$) and compared the results under varying levels of noise, as shown in Table 8. We observe that PINNs with PAFs outperform vanilla PINNs. The advantage of PAFs is even more clear in the noisier

observations. It demonstrates the flexibility of using (self-adaptive) PAFs in PINNs to accurately compute inverse parameters from noisy datasets. The use of PAFs in the presence of observational noises allows physics-based regularization of the network.

3.2.2. Boundary conditions as a PAF (Laplace Equation)

In this second inverse problem, the Laplace equation is studied, where the goal is to calculate unknown parameters of the periodic boundaries in the Cartesian scale. The Laplace Equation is defined as:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \quad (53)$$

This equation is solvable by constraining the boundaries. Here, the boundaries of the system were constrained to:

$$u(x, y; \lambda_1) = 3y \sin(y + \lambda_1); \text{ for } x = 0, \quad 2\pi \quad (54)$$

$$u(x, y; \lambda_2) = 3x \sin(x + \lambda_2); \text{ for } y = 0, \quad 2\pi \quad (55)$$

In this problem, λ_1 and λ_2 are unknown parameters whose real values need to be estimated. We generated a dataset using numerical simulation (with true values $\lambda_1 = \lambda_2 = 3.14$). From this dataset, as the observation data, we randomly separated a set of collocation points in the finite-dimensional space and on the boundaries (Fig. 10a). By considering Eqs. (54) and (55), here we defined the PAF as below:

$$\sigma_D(z, \omega_1) = 3z \sin(z + \omega_1); \quad (56)$$

where z is the output of the neuron and the ω_1 is the trainable parameter that is changed during the training of the PINN. To find the values of λ_1 and λ_2 by inverse calculation, different scenarios are evaluated using Eq. (56) in the second layer of the network. The calculation results are

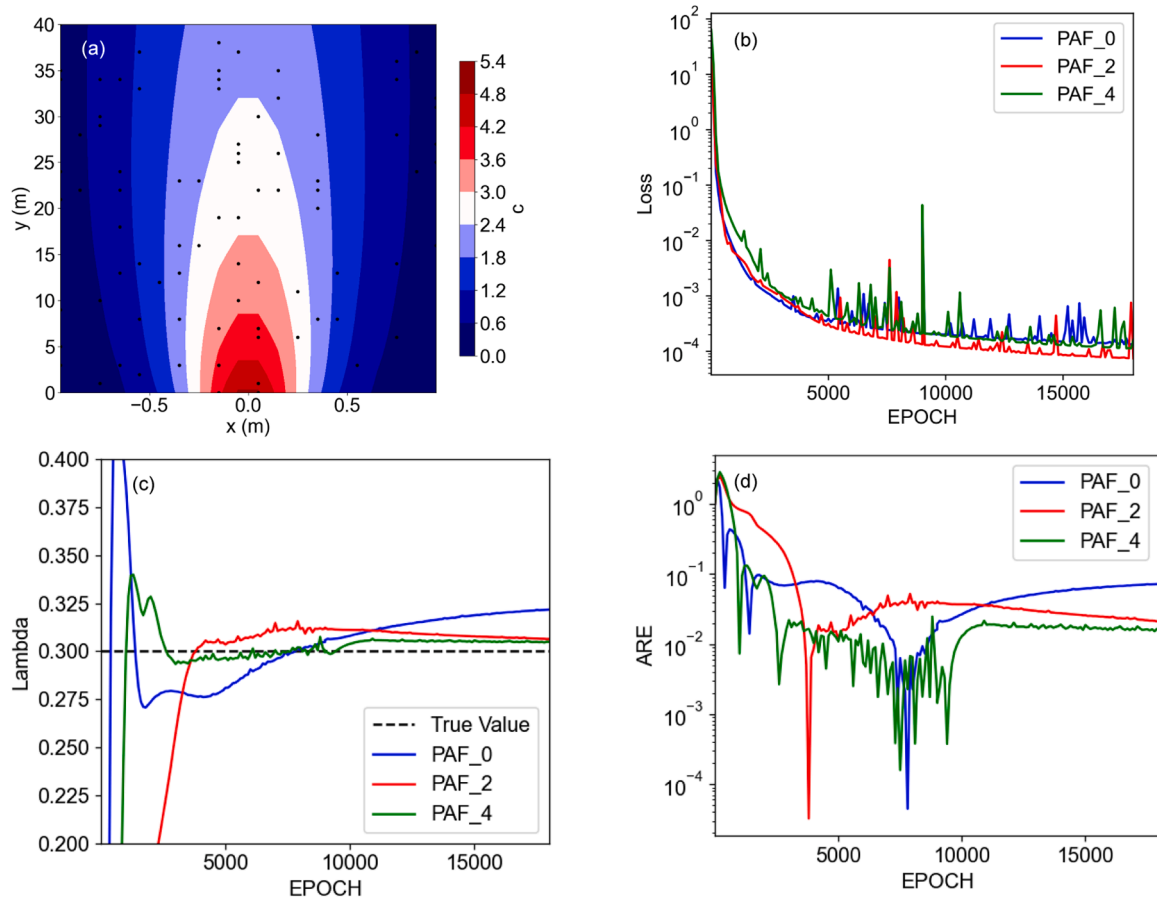


Fig. 9. The results of using PINNs for inverse calculation of diffusion coefficient parameters with and without using PAFs. a) The contour map of the diffusion process and the randomly chosen data (black dots) in the domain. a) the optimizing process done in 16000 epochs, with LR of 0.003 and 0.001, b) the value of λ_1 during optimization, c) the ARE values for the cases with and without PAF.

Table 7

The results of inverse calculations using PAFs used for obtaining the values of λ_1 and λ_2 .

MLP	Total Loss	λ_1, λ_2	ARE λ_1, λ_2
PAF_0	1.59e-04	0.322, 0.288	0.073, 0.040
PAF_2	7.41e-05	0.306, 0.289	0.020, 0.036
PAF_4	1.19e-04	0.305, 0.289	0.016, 0.036

Table 8

The ARE values of the calculated inverse parameters under varying levels of Gaussian noise in the observational data, and different number of observational collocation points. In all the cases, we used a self-adaptive approach to make sure the comparisons are reliable. In the base case (No PAF), we utilized tanh AF, and in SA-PAF cases, we applied a combination of tanh and σ_D (Eq. (52)) AFs.

Number of Observational Points	Noise Level	0.0 %	2.5 %	5.0 %	7.5 %
300	No PAF (tanh)	0.015	0.032	0.134	0.436
	SA-PAF	0.014	0.010	0.054	0.547
450	No PAF (tanh)	0.015	0.027	0.084	0.595
	SA-PAF	0.014	0.017	0.018	0.245
600	No PAF (tanh)	0.015	0.017	0.076	0.262
	SA-PAF	0.006	0.009	0.027	0.094

shown in Table 9 and Fig. 10b. We investigated different cases with different numbers of neurons that use PAFs in their structure (PAF_0 to PAF_5). We can see that the cases with 1 or 2 PAF neurons (PAF_1 or PAF_2) achieved lower loss values (otherwise the neurons were activated with the tanh AF). The calculated λ_1 values and its relative error are shown in Fig. 10c and d, and in Table 9, where the NNs with 1–3 neurons resulted in more accuracy. This example also shows how PAFs could improve the PINNs-based inverse calculations. However, the dominance of PAFs in a single layer of NNs can reduce flexibility and weaken the calculations.

4. Discussion

The idea of using PINNs in leveraging the expressiveness of NNs in scientific machine learning has received growing attention in recent years. In this work, we assessed PINNs in predicting the out-of-training-distribution testing data. The results showed that the PINNs rarely predicted acceptably for out-of-training-distribution testing data, in line with the observations of Kim et al. [18]. The most critical question is how to select an acceptable PAF. Ideal performance was achieved with terms inspired by the analytical solution of the PDE of interest. However, in problems that we usually deal with, there are no analytical solutions, and numerical approaches are solving the problems. In general, any mathematical function involved in the workflow of the solution of the investigated PDE may be tested as a PAF.

However, there are always trade-offs between the simplicity of using general AFs and the advantages of using the PAFs. Using PAFs brings many benefits, but selecting, implementing PAFs, and training the PINNs with PAFs may be challenging, as previously discussed. In addi-

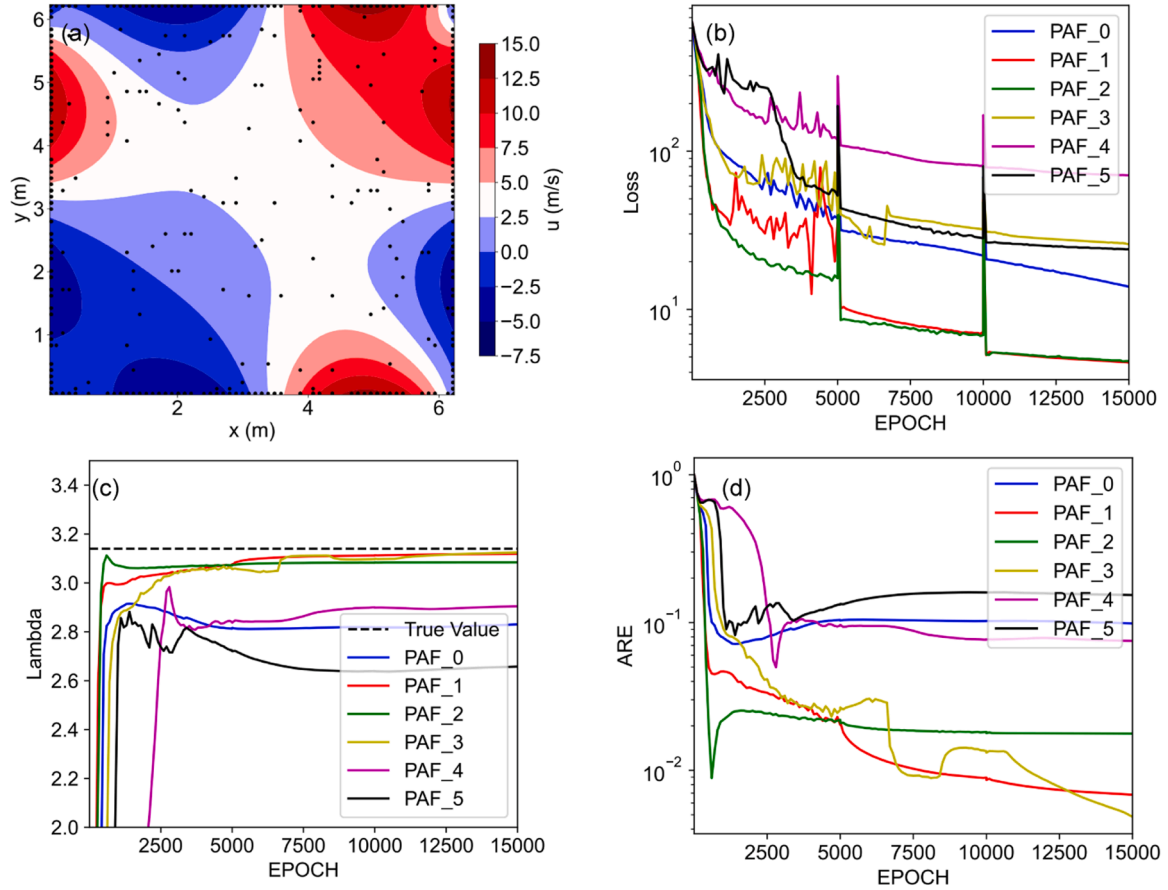


Fig. 10. Utilizing PINNs for inverse calculation of boundary condition parameters in the Laplace equation. a) The contour map of the system in 2D and the randomly chosen points in the domain and on the boundaries, b) the training process performed in 15000 epochs, with LR of 0.003–0.001, c) the value of λ_1 during optimization, d) ARE values for λ_1 during optimization.

Table 9

The results of inverse calculations used for obtaining the values of λ_1 and λ_2 in the PINNs with different numbers of neurons (0–5) with PAFs.

MLP	Total Loss	λ_1, λ_2	ARE λ_1, λ_2
PAF_0	13.8	2.83, 3.01	0.099, 0.041
PAF_1	4.61	3.12, 3.14	0.007, 0.001
PAF_2	4.71	3.08, 3.14	0.018, 0.001
PAF_3	2.57	3.12, 3.22	0.005, 0.027
PAF_4	7.04	2.90, 1.43	0.075, 0.544
PAF_5	2.39	2.65, 2.94	0.153, 0.062

tion, using PAFs requires some scientific background and knowledge about the investigating phenomena. Since the general AFs (such as $\tanh$) are somehow powerful enough to regenerate the behavior of simple functions with acceptable accuracy, the advantage of using PAFs is more impressive when the non-linearity of the PAF increases; for instance, PINNs are very weak in learning and extrapolating functions including periodic behavior. However, we realized complications when the PAFs were added to the networks. The first challenge was with the domain of the function. In general, the input to a neuron is in the domain of real numbers ($z \in \mathbb{R}$). This is while the domain of PAFs may be limited. In these cases, the training process fails. So, some actions are required to be taken to extend the domain of the function into $\mathbb{R}$. Also, in some cases, we observed that the chance of failure of the optimizer in the PAF-included PINNs was increased in comparison to the PINNs without PAFs. In some cases, these were due to the problems in the differentiability of the defined PAF in some parts of its domain. Also, sharp changes in the function slope curve may lead to instability problems in the

model. The capability of using PAFs in reducing the number of unique solutions for a PINN is a double-edged sword. Since the number of unique solutions is lower in PAF-PINNs, finding possible solutions is more challenging for the gradient descent optimizer. However, the application of self-adaptive PAFs almost resolves this challenge, as we have shown in several examples.

5. Conclusions

This work introduces a new class of activation functions called Physical Activation Functions (PAFs) that improve the physics-informed neural networks (PINNs). PAF, which can be categorized as a subgroup of inductive mathematical approaches for fusing physics with NNs, is a mathematical expression that gets inspiration from the physical model of the investigated phenomena. We demonstrated the potential of PAFs for both forward and inverse problems in different cases, i.e., 1D and 2D wave equations, advection-convection phenomenon, 1D and 2D diffusion processes, and the Laplace equation. The below points may be concluded from the results:

- Based on examples, we showed that although the performance of PINNs was significant in the training ranges, they could not predict well on out-of-training distribution ranges. Using PAFs allowed to reduce the PINN's network size by up to 75 % while keeping the solution accuracy. Furthermore, PAFs make PINNs more physically constrained. This improves prediction and reduces errors of out-of-training-distribution predictions. Additionally, using PAFs in PINNs may help reduce the training complexity of the PINNs and the final

loss values. In some examples, the total loss term is reduced by 1–2 orders of magnitude.

- The PAFs are also effective in deep PINNs when PAFs are only active in a small fraction of the neurons. However, by increasing the model's width, the impact of the PAFs was reduced in the examples of this paper. Likely, the optimizer could not allocate enough importance to the neurons using PAFs. In the more complex cases, a self-adaptive implementation approach is introduced that can be shown to be positive in improving the training process.
- PAFs could improve PINN inverse calculations for the studied cases by reducing the degree of freedom of the NN model. However, overusing the PAFs did reduce the accuracy of the inverse calculations. The application of SA-PAFs could improve the performance of PINNs in inverse computations for noisy observational datasets.
- From a theoretical point of view, the PAFs should ideally be whole or parts of the analytical solution of the PDE being studied. In practice, analytical solutions may be unavailable, and PAFs can be obtained from analytical solutions of the PDE under simpler assumptions or lower dimensions (but same temporal-spatial behavior). Moreover, it can be inspired by the initial or boundary conditions of the problem.

CRedit authorship contribution statement

Pål Andersen: Conceptualization, Funding acquisition, Supervision, Validation, Writing – review & editing. **Jassem Abbasi:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Visualization, Writing – original draft.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We will share it after acceptance of the paper.

Acknowledgments

Andersen acknowledges the Research Council of Norway and the industry partners of NCS2030 – RCN project number 331644 – for their support.

References

- [1] J. Abbasi, P.O. Andersen, Simulation and prediction of countercurrent spontaneous imbibition at early and late time using physics-informed neural networks, *Energy Fuels* 0 (0) (2023), <https://doi.org/10.1021/acs.energyfuels.3c02271>.
- [2] M.M. Almajid, M.O. Abu-Al-Saud, Prediction of porous media fluid flow using physics informed neural networks, *J. Pet. Sci. Eng.* 208 (2022), <https://doi.org/10.1016/j.petrol.2021.109205>.
- [3] C. Bergman, *Universal algebra: Fundamentals and selected topics*, Chapman and Hall/CRC, 2011.
- [4] A. Bora, W. Dai, J.P. Wilson, J.C. Boyt, Neural network method for solving parabolic two-temperature microscale heat conduction in double-layered thin films exposed to ultrashort-pulsed lasers, *Int. J. Heat. Mass Transf.* 178 (2021) 121616, <https://doi.org/10.1016/j.jheatmasstransfer.2021.121616>.
- [5] S. Cai, Z. Mao, Z. Wang, M. Yin, G.E. Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: a review, *Acta Mech. Sin. /Lixue Xuebao* (2022), <https://doi.org/10.1007/S10409-021-01148-1>.
- [6] M.J. Colbrook, V. Antun, A.C. Hansen, The difficulty of computing stable and accurate neural networks: on the barriers of deep learning and Smale's 18th problem, *Proc. Natl. Acad. Sci. USA* 119 (12) (2022), <https://doi.org/10.1073/PNAS.2107151119>.
- [7] M. Cranmer, A. Sanchez-Gonzalez, P. Battaglia, R. Xu, K. Cranmer, D. Spergel, S. Ho, Discovering Symbolic Models from Deep Learning with Inductive Biases, *Adv. Neural Inf. Proc. Syst.* (2020). <https://doi.org/10.48550/arxiv.2006.11287>.
- [8] Cuomo, S., di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., & Piccialli, F. (2022). *Scientific Machine Learning through Physics-Informed Neural Networks: Where we are and What's next.* (<http://arxiv.org/abs/2201.05624>).
- [9] Djeumou, F., Neary, C., Goubault, E., Putot, S., & Topcu, U. (2021). *Neural Networks with Physics-Informed Architectures and Constraints for Dynamical Systems Modeling.* (<https://doi.org/10.48550/arxiv.2109.06407>).
- [10] L.C. Evans, *Partial differential equations*, Vol. 19, American Mathematical Society, 2022.
- [11] O. Fuks, H.A. Tchelepi, Limitations of physics informed machine learning for nonlinear two-phase transport in porous media, *J. Mach. Learn. Model. Comput. Vol.* 1 (Issue 1) (2020). (www.begellhouse.com).
- [12] T. Hey, K. Butler, S. Jackson, J. Thyagalingam, Machine learning and big scientific data, *Philos. Trans. R. Soc. A* 378 (2166) (2020), <https://doi.org/10.1098/RSTA.2019.0054>.
- [13] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366, [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- [14] A.D. Jagtap, K. Kawaguchi, G. Em Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, *Proc. R. Soc. A: Math. Phys. Eng. Sci.* 476 (2239) (2020) 20200334, <https://doi.org/10.1098/rspa.2020.0334>.
- [15] A.D. Jagtap, Y. Shin, K. Kawaguchi, G.E. Karniadakis, Deep Kronecker neural networks: a general framework for neural networks with adaptive activation functions, *Neurocomputing* 468 (2021) 165–180. (<https://doi.org/10.48550/arxiv.2105.09513>).
- [16] X. Jin, S. Cai, H. Li, G.E. Karniadakis, NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations, *J. Comput. Phys.* 426 (2021) 109951, <https://doi.org/10.1016/j.jcp.2020.109951>.
- [17] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, in: *Nature Reviews Physics*, Vol. 3, Springer Nature, 2021, pp. 422–440, <https://doi.org/10.1038/s42254-021-00314-5>.
- [18] Kim, J., Lee, K., Lee, D., Jin, S.Y., & Park, N. (2020). DPM: A Novel Training Method for Physics-Informed Neural Networks in Extrapolation. *ArXiv Preprint ArXiv:2012.02681*.
- [19] D.P. Kingma, J.L. Ba, Adam: A Method for Stochastic Optimization, 3rd Int. Conf. Learn. Represent., ICLR 2015 - Conf. Track Proc. (2014). (<https://arxiv.org/abs/1412.6980v9>).
- [20] D. Kochkov, J.A. Smith, A. Alieva, Q. Wang, M.P. Brenner, S. Hoyer, Machine learning-accelerated computational fluid dynamics, *Proc. Natl. Acad. Sci. USA* 118 (21) (2021). (https://doi.org/10.1073/PNAS.2101784118/SUPPL_FILE/PNAS.2101784118.SAPP.PDF).
- [21] Krishnapriyan, A.S., Gholami, A., Zhe, S., Kirby, R.M., & Mahoney, M.W. (2021). *Characterizing possible failure modes in physics-informed neural networks.* (<http://arxiv.org/abs/2109.01050>).
- [22] J. Kubalik, E. Derner, R. Babuška, Multi-objective symbolic regression for physics-aware dynamic modeling, *Expert Syst. Appl.* 182 (2021) 115210, <https://doi.org/10.1016/j.eswa.2021.115210>.
- [23] Leiteritz, R., & Pflüger, D. (2021). *How to Avoid Trivial Solutions in Physics-Informed Neural Networks.* (<http://arxiv.org/abs/2112.05620>).
- [24] Li, J., Zhang, D., Wang, N., & Chang, H. (2021). *Deep Learning of Two-Phase Flow in Porous Media via Theory-Guided Neural Networks.* (<https://doi.org/10.2118/208602-PA/2556411/spe-208602-pa.pdf/1>).
- [25] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229, <https://doi.org/10.1038/S42256-021-00302-5>.
- [26] S. Markidis, The old and the new: can physics-informed deep-learning replace traditional linear solvers? *Front. Big Data* 4 (2021) 92. (<https://doi.org/10.3389/FDATA.2021.669097/BIBTEX>).
- [27] G.S. Misyris, A. Venzke, S. Chatzivasilieadis, Physics-informed neural networks for power systems, *IEEE Power and Energy Society General Meeting*, 2020-August (2020), <https://doi.org/10.1109/PESGM41954.2020.9282004>.
- [28] E. Mjølness, D. DeCoste, Machine learning for science: State of the art and future prospects, *Science* 293 (5537) (2001) 2051–2055. (<https://doi.org/10.1126/SCIENCE.293.5537.2051/ASSET/59CAD30D-7FF2-4A1E-ACBC-8B3DDDF483AC/ASSETS/GRAPHIC/SE3619767002.JPEG>).
- [29] Moseley, B., Markham, A., & Nissen-Meyer, T. (2020). *Solving the wave equation with physics-informed deep learning.* (<https://doi.org/10.48550/arxiv.2006.11894>).
- [30] Peng, W., Zhou, W., Zhang, J., & Yao, W. (2020). *Accelerating Physics-Informed Neural Network Training with Prior Dictionaries.* (<https://doi.org/10.48550/arxiv.2004.08151>).
- [31] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707, <https://doi.org/10.1016/j.jcp.2018.10.045>.
- [32] P. Ramachandran, B. Zoph, Q. v le Google Brain, Searching for activation functions, 6th International Conference on Learning Representations, ICLR 2018 - Workshop Track Proceedings (2017). (<https://doi.org/10.48550/arxiv.1710.05941>).
- [33] M.W. Scroggs, J.S. Dokken, C.N. Richardson, G.N. Wells, Construction of arbitrary order finite element degree-of-freedom maps on polygonal and polyhedral cell meshes, *ACM Trans. Math. Softw. (TOMS)* 48 (2) (2022), <https://doi.org/10.1145/3524456>.
- [34] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, *J. Chem. Phys.* 128 (13) (2008) 134106, <https://doi.org/10.1063/1.2841941>.
- [35] Y. Shin, J. Darbon, G.E. Karniadakis, On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs (2020), <https://doi.org/10.4208/cicp.OA-2020-0193>.

- [36] K. Shukla, A.D. Jagtap, G.E. Karniadakis, Parallel physics-informed neural networks via domain decomposition, *J. Comput. Phys.* **447** (2021) 110683, <https://doi.org/10.1016/J.JCP.2021.110683>.
- [37] H. Wang, L. Lu, S. Song, G. Huang, Learning Specialized Activation Functions for Physics-informed Neural Networks, *ArXiv Prepr. ArXiv:2308.04073* (2023).
- [38] Z. Xiang, W. Peng, X. Liu, W. Yao, Self-adaptive loss balanced Physics-informed neural networks, *Neurocomputing* **496** (2022) 11–34, <https://doi.org/10.1016/J.NEUCOM.2022.05.015>.
- [39] Zheng, C., & Bennett, G. (2002). *Applied contaminant transport modeling*. (http://sutlib2.sut.ac.th/sut_contents/H74178.pdf).
- [40] L. Ziyin, T. Hartwig, M. Ueda, Neural networks fail to learn periodic functions and how to fix it, *Adv. Neural Inf. Process. Syst.* **33** (2020) 1583–1594.
- [41] Zubov, K., McCarthy, Z., Ma, Y., Calisto, F., Pagliarino, V., Azeglio, S., Bottero, L., Luján, E., Sulzer, V., Bharambe, A., Vinchhi, N., Balakrishnan, K., Upadhyay, D., & Rackauckas, C. (2021). *NeuralPDE: Automating Physics-Informed Neural Networks (PINNs) with Error Approximations*.
- [42] D. Zwicker, *py-pde: A python package for solving partial differential equations*, *J. Open Source Softw.* **5** (48) (2020) 2158.



Jassem Abbasi is PhD candidate in Petroleum Technology in Department of Energy Resources at University of Stavanger. He focuses on the application of Physics Informed Neural Networks in improving the prediction of physical processes, especially flow in porous media. Before starting his PhD, for almost 4 years, he worked with different institutions/ companies as both researcher and engineer in different technical disciplines such as reservoir engineering, reservoir simulation, enhanced oil recovery (EOR), reservoir fluid properties, and software development.



Pål Østebø Andersen is Associate Professor at University of Stavanger and holds a PhD in Petroleum Technology. He has authored over 80 technical papers and received the 2021 SPE Cedric K Ferguson Medal. His main research interests include simulation, special core analysis, design and interpretation of experiments, machine learning and the energy transition.